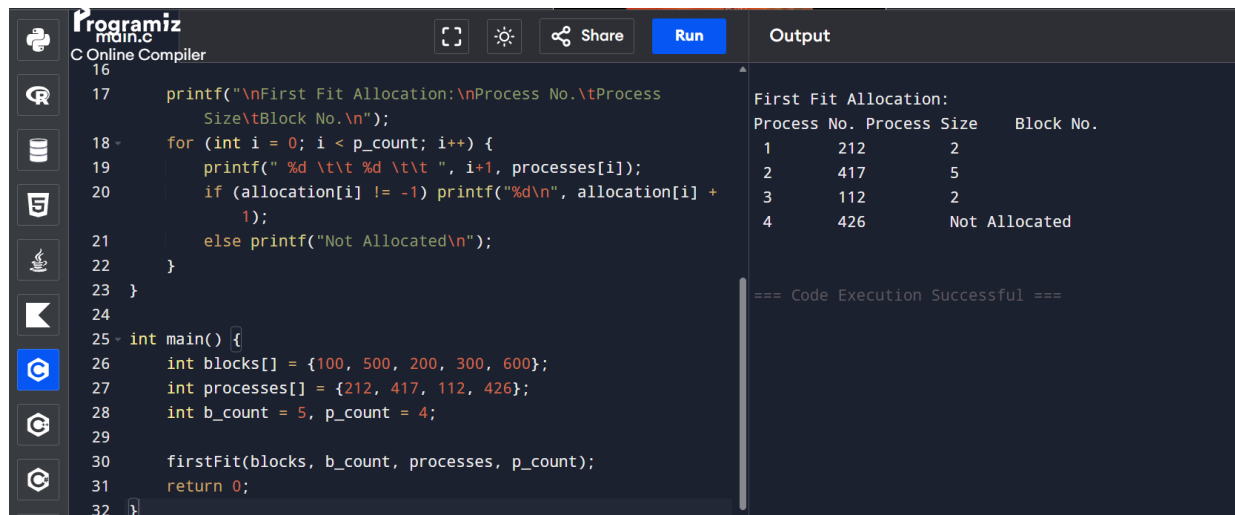


EXPERIMENT 13:

13. Construct a C program for implementation the various memory allocation strategies. .



The screenshot shows a C program in an online compiler. The code implements the First Fit memory allocation strategy. It defines an array of block sizes and an array of process sizes. The program iterates through each process and attempts to allocate it to the first block that is large enough. If a process cannot be allocated, it prints "Not Allocated". The output shows the allocation results for four processes.

```
16
17     printf("\nFirst Fit Allocation:\nProcess No.\tProcess
    Size\tBlock No.\n");
18     for (int i = 0; i < p_count; i++) {
19         printf(" %d \t\t %d \t\t ", i+1, processes[i]);
20         if (allocation[i] != -1) printf("%d\n", allocation[i] +
    1);
21         else printf("Not Allocated\n");
22     }
23 }
24
25 int main() {
26     int blocks[] = {100, 500, 200, 300, 600};
27     int processes[] = {212, 417, 112, 426};
28     int b_count = 5, p_count = 4;
29
30     firstFit(blocks, b_count, processes, p_count);
31     return 0;
32 }
```

Output:

```
First Fit Allocation:
Process No. Process Size   Block No.
1          212           2
2          417           5
3          112           2
4          426          Not Allocated

=== Code Execution Successful ===
```

PSEUDO CODE :

1. Input block sizes and process sizes.
2. First Fit: Allocate process to the FIRST block large enough.
3. Best Fit: Allocate process to the SMALLEST block large enough.
4. Worst Fit: Allocate process to the LARGEST block available.
5. Print allocation result for each process.

CODE:

```
#include <stdio.h>

void firstFit(int blocks[], int b_count, int processes[], int p_count) {

    int allocation[10];

    for(int i = 0; i < p_count; i++) allocation[i] = -1;

    for (int i = 0; i < p_count; i++) {

        for (int j = 0; j < b_count; j++) {

            if (blocks[j] >= processes[i]) {

                allocation[i] = j;

                blocks[j] -= processes[i];

                break;

            }

        }

    }

    printf("\nFirst Fit Allocation:\nProcess No.\tProcess Size\tBlock No.\n");

    for (int i = 0; i < p_count; i++) {

        printf(" %d \t\t %d \t\t ", i+1, processes[i]);

        if (allocation[i] != -1) printf("%d\n", allocation[i] + 1);

        else printf("Not Allocated\n");

    }

}

int main() {

    int blocks[] = {100, 500, 200, 300, 600};

    int processes[] = {212, 417, 112, 426};
```

```
int b_count = 5, p_count = 4;
```

```
firstFit(blocks, b_count, processes, p_count);
```

```
return 0;
```

```
}
```